RFPO & PO Change Management Plan

Executive Summary

This document outlines a comprehensive plan for enabling two change management capabilities within the RFPO application:

1. **Scenario A** — Modifying an RFPO or PO **after** the approval process has started or after a PO has been generated
2. **Scenario B** — Changing approvers in the approval chain **after** the RFPO has been submitted for approval

Both scenarios carry significant governance risks. This plan analyzes the current system, identifies gaps, proposes solutions with built-in safeguards, and surfaces questions that must be answered by the customer before implementation.

> **Reviewed by:** Workflow Systems Expert (state machine, concurrency, multi-phase) and Approval Process & Procurement Governance Expert (fraud prevention, audit, compliance). Key findings from both reviews are incorporated throughout, marked with "REVIEWER FINDING."

Current System Analysis

How the Approval Workflow Works Today

```
┌──────────────────────────────────────────────────────────────────┐
│  WORKFLOW TEMPLATE (RFPOApprovalWorkflow)                        │
│  - Defined per Consortium, Team, or Project                      │
│  - Contains Stages (budget brackets) → Steps (approvers)         │
│  - Only ONE active template per entity at a time                 │
└──────────────────────────────────────────────────────────────────┘
        │
        │  On RFPO submission
        ▼
┌──────────────────────────────────────────────────────────────────┐
│  WORKFLOW INSTANCE (RFPOApprovalInstance)                         |
│  - 1:1 relationship with RFPO (uselist=False)                    │
│  - Snapshots the workflow template into `instance_data` JSON     │
│  - Tracks current_stage_order + current_step_order               │
│  - overall_status: draft → waiting → approved/refused            │
└──────────────────────────────────────────────────────────────────┘
        │
        │  Sequential gating
        ▼
┌──────────────────────────────────────────────────────────────────┐
│  APPROVAL ACTIONS (RFPOApprovalAction)                           │
│  - Created one-at-a-time (sequential, not parallel)              │
│  - Each action records: stage/step order, approver, status       │
│  - Snapshots approver name, stage name, step name at creation    |
```

```
│  - Status: pending → approved/conditional/refused                │
│  - On approve: _create_next_sequential_action() fires            │
│  - On refuse: entire workflow immediately marked "refused"       │
└──────────────────────────────────────────────────────────────────┘
        │
        │  All actions approved
        ▼
┌──────────────────────────────────────────────────────────────────┐
│  PO GENERATED                                                    │
│  - PO number assigned: PO-{ABBREV}-{YYYYMMDD}-{SEQ}             │
│  - RFPO status set to "Approved"                                 │
│  - PO PDF can be generated from RFPO + consortium template       │
└──────────────────────────────────────────────────────────────────┘
```

Multi-Phase Sequential Approval

The system supports up to 3 sequential phases, determined at submission:

1. **Phase 1: Project-specific workflow** (if `project_id` exists and has active workflow)
2. **Phase 2: Team-specific workflow** (if `team_id` exists and has active workflow)
3. **Phase 3: Consortium workflow** (always applied if `consortium_id` exists)

Phases are fully sequential — Phase 2 doesn't start until all of Phase 1's steps complete.

Key Current Constraints

Constraint	Detail
1:1 Instance	An RFPO can only have ONE approval instance (`uselist=False`). The system rejects submission if one already exists.
Snapshot-based	The `instance_data` JSON is a point-in-time snapshot. Changing the template does NOT affect in-flight instances.
Sequential actions	Actions are created one-at-a-time. Future approvers don't have an action record yet.
No versioning on RFPO	No revision history, version number, or change log. Edits overwrite data.
PO number is permanent	Assigned once on final approval. No concept of PO revision or amendment.
No audit trail on data changes	`updated_at`/`updated_by` exist but there's no record of *what* changed.
No post-approval lock	Nothing technically prevents editing an RFPO after it's been approved.
Parallel stages supported but unused	The `is_parallel` flag exists on stages but current logic creates actions sequentially.

Scenario A: Modifying an RFPO/PO After Approval Starts

The Problem

If an RFPO is changed after approvers have already approved it, the approval no longer represents what was approved. Approver #1 approved "$50,000 for Widget A" but the RFPO now says "$75,000 for Widget B."

Proposed Solution: RFPO Amendment Workflow

Concept: "Amendments, Not Edits"

Changes to an in-flight or approved RFPO should not silently overwrite data. Instead, the system creates a formal **Amendment** that records what changed, invalidates affected approvals, and restarts the process as needed.

Pre-Requisite: RFPO Locking

> **REVIEWER FINDING (Governance):** The current system has no protection against post-approval edits. An admin could edit the RFPO after all approvals complete but before PO generation, silently invalidating every approval.

Before any amendment workflow is built, implement RFPO locking:

- Add `is_locked` flag to RFPO model (default `False`)
- When approval instance reaches `overall_status = "approved"`: set `rfpo.is_locked = True`
- Before any RFPO edit in admin panel: check lock status, reject unless going through amendment workflow
- RFPO_ADMIN can force-unlock ONLY through the amendment workflow (audit-logged, reason required)

Pre-Requisite: RFPO Snapshot on Approval Actions

> **REVIEWER FINDING (Governance):** The current system does NOT bind an approval to the RFPO data state at the time of action. There's no record of what each approver actually saw when they approved.

Add `rfpo_snapshot_json` (TEXT) to `RFPOApprovalAction` — captures total amount, subtotal, vendor, line item count at the moment the action is **created**. This enables forensic queries: "What did Approver X see when they approved?"

New Data Model: RFPOAmendment

```
┌──────────────────────────────────────────────┐
│  RFPOAmendment (NEW)                         │
│  ────────────────────────────                │
│  id (PK)                                     │
│  amendment_id (e.g., "AMD-001")              │
│  rfpo_id (FK → RFPO)                         │
│  amendment_number (1, 2, 3...)               │
│  reason (TEXT - REQUIRED)                    │
│  change_summary (TEXT)                       │
│  change_detail (JSON - field-level diff)     │
│  previous_snapshot (JSON - full RFPO state)  │
```

```
│  new_snapshot (JSON - proposed new state)    │
│  requested_by (user record_id)               │
│  approved_by (user record_id, nullable)      │
│  status (pending/approved/rejected/applied)  │
│  amendment_type (minor/major/critical)       │
│  affected_phases (JSON - e.g., [2,3])        │
│  impacts_total_amount (BOOL)                 │
│  previous_total (Numeric)                    │
│  new_total (Numeric)                         │
│  amendment_status_log (JSON - append-only)   │
│  created_at / updated_at                     │
└──────────────────────────────────────────────┘
```

The `amendment_status_log` captures the full lifecycle as an append-only record:

```
[
  {"status": "pending", "at": "2026-04-03T10:00", "by": "requestor@co.com"},
  {"status": "approved", "at": "2026-04-03T11:30", "by": "admin@co.com",
"conditions": "..."},
  {"status": "applied", "at": "2026-04-03T12:00", "by": "system"}
]
```

Amendment Classification

Type	Trigger	Effect on Approvals
Minor	Non-financial changes: shipping address, delivery date, requestor contact, description	Notify all prior approvers. **Do NOT reset approvals.** 5-business-day objection window.
Major	Financial changes: line items, quantities, prices, vendor, any total amount change	**Reset affected phase approvals.** Phase-aware restart. All prior approvers notified.
Critical	Vendor change when total > $50K, total amount change > 20%, fundamental scope change	**Reset ALL phase approvals + require GOD-level authorization** to even initiate.

Phase-Aware Amendment Reset

> **REVIEWER FINDING (Workflow):** Resetting all phases indiscriminately wastes time. If Phases 1-2 are complete and the amendment only impacts Phase 3 concerns, forcing re-approval of Phase 1-2 creates unnecessary friction.

```
AMENDMENT RESET LOGIC:

  IF amendment impacts budget bracket (changes Stage selection):
    → Re-evaluate ALL phases against new budget bracket
    → If Stage changes in any phase: restart from that phase forward
```

```
    → All phases after the affected phase also restart

  IF amendment impacts vendor (but not budget bracket):
    → Restart from Phase 1 (vendor affects all levels)

  IF amendment impacts non-financial fields (minor):
    → No phase resets; notify-and-objection-window only

  IF amendment impacts delivery/logistics only:
    → Restart from Consortium phase only (Phase 3 typically)
    → Earlier phase approvals stand
```

The `affected_phases` JSON array on `RFPOAmendment` records which phases are reset.

Handling the 1:1 Instance Constraint

> **REVIEWER FINDING (Workflow):** The `uselist=False` on `approval_instance` prevents having both a superseded instance and a new active instance. This is a **blocking architectural issue** that must be resolved first.

Required change: Convert from 1:1 to 1:many relationship:

```python
# RFPO model change
approvals = db.relationship(
    "RFPOApprovalInstance",
    backref=db.backref("rfpo", uselist=False),
    lazy=True,
    cascade="all, delete-orphan",
    order_by="RFPOApprovalInstance.created_at.desc()"
)

# Helper methods on RFPO
def get_active_approval_instance(self):
    return next(
        (i for i in self.approvals
         if i.overall_status not in ["superseded", "voided"]),
        None
    )

def get_approval_history(self):
    return [i for i in self.approvals
            if i.overall_status in ["superseded", "voided"]]
```

Add to `RFPOApprovalInstance`:

- `superseded_by_id` (FK → self, nullable) — links to replacement instance
- `amendment_id` (FK → RFPOAmendment, nullable) — links to triggering amendment
- New `overall_status` value: `"superseded"`

Concurrency Safeguard

> **REVIEWER FINDING (Workflow):** Race condition — approver opens form at 2:00 PM, admin supersedes instance at 2:05 PM, approver submits at 2:10 PM against stale instance.

Add check to `approval_action_approve()`:

```
if instance.overall_status == "superseded":
    flash("This approval workflow has been superseded by an amendment. "
          "Please review the new workflow instance.", "error")
    return redirect(...)
```

PO Number Handling After Amendment

> **REVIEWER FINDING (Governance):** PO revision suffixes (e.g., `-R1`) create accounting system chaos. Most ERPs treat the suffix as a different PO entirely, causing reconciliation failures.

Scenario	Recommended Approach
Amendment **before** PO generated	No PO number exists yet. No impact.
Minor amendment after PO generated	Keep same PO number. Issue separate **PO Amendment Document** referencing original.
Major amendment after PO generated	**Void original PO** → generate new PO number. Memo: "Replaces PO-USCAR-20260402-001 (voided)."

> **QUESTION FOR CUSTOMER:** Does your accounting/ERP system support PO revisions natively, or must we void-and-reissue?

> **QUESTION FOR CUSTOMER:** When a PO has been sent to a vendor, what is the current process for changes?

Amendment Initiation Authority

> **REVIEWER FINDING (Governance):** If the requestor can initiate amendments, they can repeatedly amend to find an approver who accepts — classic "shopping for approval" fraud.

Rule: Requestor CANNOT directly initiate amendments. They can *request* an amendment, but an admin must authorize the amendment itself before it's created.

Requester	Can Initiate?	Authorization Required?
RFPO Requestor	**No** — can REQUEST only, admin must authorize	Admin approval of the request
Project Manager	Yes (for project-level RFPOs)	Finance admin co-authorization
RFPO_ADMIN	Yes	Different RFPO_ADMIN or GOD must authorize

Requester	Can Initiate?	Authorization Required?
GOD-level user	Yes	Self-authorize (logged)

> **QUESTION FOR CUSTOMER:** Should the requestor be completely blocked from suggesting amendments, or able to submit a request for admin review?

Amendment Limits

Policy	Rule
Count limit	Max 3 amendments per RFPO before mandatory cancellation and re-submission
Time limit	No amendments after 30 days from final approval
Financial threshold	Cumulative changes exceeding 20% of original total require GOD authorization
Admin override	GOD can override all limits (audit-logged with reason)

> **QUESTION FOR CUSTOMER:** Are these limits acceptable, or do you need different thresholds?

Scenario B: Changing Approvers Mid-Process

The Problem

The approval chain is snapshotted into `instance_data` at submission time. Approvers are hardcoded by `record_id`. Changing who approves means changing the snapshot. Two risks:

1. **Gaming risk**: Requestor knows Approver X will reject, swaps in Approver Y who will rubber-stamp it
2. **Integrity risk**: Workflow was designed to require specific people — swapping undermines the control

Current Approver Selection Architecture

```
Workflow Template (admin-defined)
  └── Stage (budget bracket)
       └── Step
            ├── primary_approver_id  (specific user)
            ├── backup_approver_id   (specific user, optional)
            └── approval_type_key    (e.g., "Technical Review")
                                      ↑
                                      This is the ROLE, not the person
```

At submission, these are copied into the `instance_data` JSON snapshot. Only the primary or backup approver from the snapshot can take action.

Proposed Solution: Controlled Approver Substitution

Concept: "Substitution, Not Replacement"

Formal substitution with mandatory dual authorization, role validation, and cooling-off period.

New Data Model: RFPOApproverSubstitution

```
┌──────────────────────────────────────────┐
│  RFPOApproverSubstitution (NEW)          │
│  ──────────────────────────────          │
│  id (PK)                                 │
│  substitution_id (e.g., "SUB-001")       │
│  instance_id (FK → RFPOApprovalInstance) │
│  action_id (FK → RFPOApprovalAction)     │
│  stage_order / step_order (INT)          │
│  original_approver_id (record_id)        │
│  substitute_approver_id (record_id)      │
│  reason (TEXT - REQUIRED)                │
│  requested_by (record_id)                │
│  authorized_by (record_id - DIFFERENT user) │
│  status (pending | authorized | activated | │
│          objected | rejected | cancelled) │
│  constraints_met (JSON - validation results) │
│  authorized_at (DateTime)                │
│  activation_time (DateTime - after cooling) │
│  objection_deadline (DateTime)           │
│  created_at / updated_at                 │
└──────────────────────────────────────────┘
```

New Data Model: ApproverPool (Role-Based Validation)

> **REVIEWER FINDING (Governance):** Without role-based validation, an admin can substitute anyone for any step—including themselves—rendering all safeguards meaningless.

```
┌──────────────────────────────────────────┐
│  ApproverPool (NEW)                      │
│  ──────────────────────────              │
│  id (PK)                                 │
│  approval_type_key (FK → Lists type)     │
│  user_id (User record_id)                │
│  consortium_id (nullable - global or scoped) │
│  is_active (BOOL)                        │
│  certified_until (Date - allows expiry)  │
│  created_at / updated_at / created_by    │
└──────────────────────────────────────────┘
```

When a substitution is requested, the system queries `ApproverPool` to verify the substitute is certified for the required `approval_type_key`. If not found or expired: **reject substitution**.

> **QUESTION FOR CUSTOMER:** Should we create an explicit ApproverPool that admins maintain, or derive eligibility from existing permissions/roles?

Substitution Safeguards (10 Required Checks)

#	Safeguard	Description
1	**Role-based validation**	Substitute must be in `ApproverPool` for same `approval_type_key` with valid certification
2	**Not the requestor**	Substitute cannot be the RFPO creator (conflict of interest)
3	**Not a prior approver**	Substitute cannot have already approved an earlier step in same instance (separation of duties)
4	**Dual authorization**	A DIFFERENT admin must authorize (not the person who requested the substitution)
5	**Original approver notified**	Person being replaced always notified with objection window
6	**Full audit trail**	Every substitution logged: who requested, who authorized, why, constraints validated
7	**No post-action substitution**	If approver already acted (approved/refused), action stands. Cannot retroactively replace.
8	**24-hour cooling-off**	Between authorization and activation. Original approver can object during window.
9	**No substitution on escalated actions**	If action already escalated to backup, cannot also substitute primary
10	**Frequency cap**	Max 2 substitutions per step; max 50% of steps in a workflow can use substitutes

Who Can Request a Substitution?

Requester	Allowed?	Authorization Required?
RFPO Requestor	**No** — conflict of interest	N/A
RFPO_ADMIN	Yes	Different RFPO_ADMIN or GOD must authorize
GOD-level user	Yes	Self-authorize (logged)
Original approver (self-delegation)	Yes	Auto-authorized; 2-hour cooling-off + manager notified
Backup approver	N/A — can already act per existing design	N/A

> **QUESTION FOR CUSTOMER:** Should the requestor EVER be able to request an approver change? (Recommendation: No)

> **QUESTION FOR CUSTOMER:** If the original approver self-delegates, should there be restrictions on who they can delegate to?

Substitution Objection Workflow

```
1. Admin authorizes substitution → status = "authorized"
2. Original approver receives email with 24-hour objection window
3. If objection received within window:
   - Status = "objected"
   - Escalated to GOD-level admin for resolution
   - Substitute CANNOT act until resolved
4. GOD admin can:
   - Uphold objection → reverse substitution
   - Override objection → proceed (audit-logged)
5. If no objection within 24 hours:
   - Status = "activated"
   - New approver can act
```

Instance Snapshot Update on Substitution

> **REVIEWER FINDING (Workflow):** If `instance_data` JSON isn't updated, `_create_next_sequential_action()` creates future actions with the OLD approver ID.

When substitution activates:

1. Create an **immutable mutation record** (see below)
2. Update `instance_data` JSON: change `primary_approver_id` in affected step
3. Update pending `RFPOApprovalAction.approver_id` to the substitute

Instance Mutation Audit Log

> **REVIEWER FINDING (Governance):** `instance_data` should ideally be immutable. Use a mutation-log approach to track all changes while preserving the original.

```
┌──────────────────────────────────────────────┐
│  RFPOApprovalInstanceMutation (NEW)          │
│  ──────────────────────────────              │
│  id (PK)                                     │
│  mutation_id (e.g., "MUT-001")               │
│  instance_id (FK → RFPOApprovalInstance)     │
│  mutation_type (substitution | amendment |    │
│    escalation)                                │
│  mutation_data (JSON - the changed fields)   │
│  previous_state (JSON - before change)       │
│  mutated_by (user record_id)                 │
│  mutated_at (DateTime)                       │
│  reason_code (approver_unavailable |         │
│    budget_correction | etc.)                 │
│  created_at                                  │
└──────────────────────────────────────────────┘
```

Enables reconstructing exact workflow state at any point by replaying mutations on the original snapshot.

Backup Approver Tracking Improvement

> **REVIEWER FINDING (Governance):** Backup approvers can currently act without any substitution request. This intended governance bypass should be explicitly classified.

Add `action_pathway_type` field to `RFPOApprovalAction`:

- `normal` — primary approver acted
- `escalation` — backup acted due to timeout auto-escalation
- `delegation` — explicit self-delegation by primary
- `substitution` — admin-authorized approver substitution

Combined Workflow: Amendment + Approver Change

```
RFPO needs amendment AND approver change?
  │
  ├── Amendment is MAJOR
  │     → Full re-approval for affected phases
  │     → Admin updates workflow TEMPLATE before re-submission
  │     → No substitution needed (new instance uses updated template)
  │
  └── Amendment is MINOR + approver change needed
        → Process substitution first (24-hour cooling-off)
        → Then apply minor amendment
        → Both changes logged independently
```

Database Schema Changes Summary

New Models (4)

1. **`RFPOAmendment`** — change history with field-level diffs and append-only status log
2. **`RFPOApproverSubstitution`** — approver changes with dual authorization and cooling-off
3. **`ApproverPool`** — role-based approver certification per approval type
4. **`RFPOApprovalInstanceMutation`** — immutable mutation log for instance changes

Modified Models

Model	Changes
RFPO	Add: `version` (INT), `po_revision` (INT), `is_locked` (BOOL), `amendment_count` (INT)
RFPOApprovalInstance	Add: `superseded_by_id` (FK → self), `amendment_id` (FK → RFPOAmendment). Change to 1:many. Add `"superseded"` status.

Model	Changes
RFPOApprovalAction	Add: `rfpo_snapshot_json` (TEXT), `action_pathway_type` (STRING), `substitution_id` (FK), `approver_email` (STRING)

New Status Values

Status	Applies To	Meaning
`Amendment Pending`	RFPO	Has a pending major amendment awaiting authorization
`Superseded`	ApprovalInstance	Replaced by new instance due to amendment
`Voided`	RFPO (PO lifecycle)	PO voided due to major amendment
`PO Generated`	RFPO	Approval complete, PO number assigned

Migration Notes

- **All changes are additive** — `ALTER TABLE ADD COLUMN` only, no drops, no data loss
- **The 1:1 → 1:many change** on `approval_instance` is the most impactful refactor — all code accessing `rfpo.approval_instance` must be updated to `rfpo.get_active_approval_instance()`

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Approver gaming	HIGH without safeguards	HIGH	Requestor blocked; dual authz; role-matching; cooling-off; frequency caps
Amendment cycle abuse	HIGH without safeguards	HIGH	Requestor cannot initiate; max 3 amendments; financial threshold; time limit
Retroactive post-approval edits	MEDIUM	HIGH	RFPO locking on approval; edits only via amendment workflow
Race condition (approve on superseded instance)	MEDIUM	MEDIUM	Superseded-instance check before processing actions
Vendor confusion from PO changes	MEDIUM	HIGH	Void-and-reissue (not revision suffixes); vendor acknowledgment
Accounting system mismatch	MEDIUM	HIGH	Define ERP sync points; confirmation before PO release
Snapshot corruption	LOW	HIGH	JSON schema validation; fail-safe rejection
Audit forensics failure	LOW with proper implementation	HIGH	RFPO snapshots on actions; immutable mutation log; append-only status logs

---

Open Questions for Customer

Amendment Questions

1. **Who can initiate an amendment?** (Rec: Admins only; requestor can submit request for admin review)
2. **What constitutes "minor" vs "major"?** (Rec: Any financial field change = major; else = minor)
3. **After a PO is issued to a vendor, what is the current process for changes?** Void-and-reissue? Amendment letter?
4. **Should there be a maximum number of amendments?** (Rec: 3 before mandatory re-submission)
5. **For minor amendments, do existing approvals stand?** (Rec: Yes, with 5-day objection window)
6. **Is there a time limit after which amendments are no longer allowed?** (Rec: 30 days post-approval)

Approver Change Questions

7. **Should the RFPO requestor ever be able to request an approver change?** (Rec: No)
8. **Should there be a defined pool of eligible substitutes per approval type?** (Rec: Yes — ApproverPool)
9. **If the original approver self-delegates, are there restrictions?** (Rec: Must be in same approval type pool)
10. **Should the system support temporary bulk delegation** (e.g., vacation coverage)? (Rec: Phase 2 feature)
11. **What happens if an approver leaves the organization mid-workflow?** (Rec: Auto-escalate to backup; if no backup, admin intervention)
12. **Should approver changes be allowed after final approval?** (Rec: No — PO already issued)

General Questions

13. **What is the customer's compliance/audit framework?** (SOX, government procurement, internal?)
14. **Is there an external ERP/accounting system that receives PO data?** (Impacts revision strategy)
15. **Should amendment/substitution history be visible to all users or only admins?** (Rec: Facts visible to all; decision details admin-only)

Recommended Organizational Policies

> **REVIEWER FINDING (Governance):** Technical implementation without accompanying organizational policy creates "controls theater" — the system has safeguards but nobody knows when to use them.

Policy 1: Amendment Authority

- Requestor cannot initiate amendments (can submit request to admin)
- Minor amendments authorized by any RFPO_ADMIN
- Major amendments require different RFPO_ADMIN or GOD authorization
- Critical amendments require GOD only
- Max 3 amendments per RFPO; max 30 days from approval
- All amendments require business justification (mandatory reason field)

Policy 2: Approver Substitution

- Only RFPO_ADMIN, GOD, or original approver can request substitution
- Dual authorization required (different admin must approve)
- 24-hour cooling-off minimum between authorization and activation
- Substitute must be certified in ApproverPool for the approval type
- Max 2 substitutions per step; max 50% of steps per workflow
- Original approver has objection rights during cooling-off

Policy 3: Approval Data Integrity

- Post-approval RFPOs are read-only (locked)
- All edits post-lock require amendment workflow
- Approval actions capture RFPO financial state at assignment
- All audit records are immutable (append-only)
- Reason codes required on all state changes

Implementation Phases

Phase 0: Schema Foundation

- ☐ Add `version`, `po_revision`, `is_locked`, `amendment_count` to RFPO model
- ☐ Create `RFPOAmendment` model
- ☐ Create `RFPOApproverSubstitution` model
- ☐ Create `ApproverPool` model
- ☐ Create `RFPOApprovalInstanceMutation` model
- ☐ Add `rfpo_snapshot_json`, `action_pathway_type`, `approver_email` to RFPOApprovalAction
- ☐ Add `superseded_by_id`, `amendment_id` to RFPOApprovalInstance
- ☐ Change approval_instance from 1:1 to 1:many relationship
- ☐ Update all code accessing `rfpo.approval_instance` → `rfpo.get_active_approval_instance()`
- ☐ Migration scripts (ALTER TABLE only — no drops)
- ☐ **Validation: All existing workflows continue unchanged**

Phase 1: RFPO Locking + Minor Amendments

- ☐ Implement RFPO locking on final approval
- ☐ RFPO snapshot capture on any edit attempt
- ☐ Diff computation engine (field-level changes)
- ☐ Minor amendment creation UI in admin panel
- ☐ Amendment classification engine (minor/major/critical)
- ☐ Email notifications to prior approvers
- ☐ 5-day objection window for minor amendments
- ☐ Audit log in RFPOAmendment
- ☐ **Test: Minor amendment doesn't reset workflow**
- ☐ **Test: Locked RFPO rejects direct edits**

Phase 2: Major Amendments + Workflow Reset

- ☐ Phase-aware amendment reset logic

- ☐ Void current instance → create new instance flow
- ☐ Superseded instance check in approve flow
- ☐ PO void-and-reissue logic
- ☐ Amendment authorization workflow (dual-admin for major)
- ☐ Admin dashboard: pending amendments
- ☐ **Test: Multi-phase partial reset**
- ☐ **Test: Race condition protection**

Phase 3: Approver Substitution

- ☐ ApproverPool admin UI (certify users per approval type)
- ☐ Substitution request API + admin UI
- ☐ All 10 safeguard validations
- ☐ 24-hour cooling-off enforcement
- ☐ Objection workflow
- ☐ Instance snapshot + mutation log updates
- ☐ Self-delegation flow
- ☐ Email notifications to all parties
- ☐ **Test: Requestor cannot request substitution**
- ☐ **Test: Substitute must be in ApproverPool**
- ☐ **Test: Cannot substitute after action taken**

Phase 4: Integration & Polish

- ☐ User App visibility (amendment history, substitution info)
- ☐ Reporting: amendments per RFPO, substitution frequency, approval cycle time
- ☐ Forensic audit query validation
- ☐ Admin dashboard: pending substitutions, trends
- ☐ PDF generator: amendment number / PO replacement note
- ☐ End-to-end testing
- ☐ User training materials

Future: Enhanced Features

- ☐ Temporary bulk delegation (vacation coverage)
- ☐ Approval SLA tracking & alerts
- ☐ Vendor acknowledgment workflow for PO changes
- ☐ ERP integration (synchronization points)
- ☐ 2FA for high-value approval actions

Technical Notes

- **No breaking changes**: All features are additive. Existing RFPOs and workflows continue unchanged.
- **Migration safety**: `ALTER TABLE ADD COLUMN` only — no drops, no data loss.
- **The 1:1 → 1:many change** on `approval_instance` is the most impactful refactor. Search all code references before starting.

- **The `instance_data` JSON snapshot** is both a strength (immutable record) and a constraint. The mutation log preserves immutability while allowing operational changes.
- **Parallel stage support**: Design amendment logic to work with both sequential and parallel patterns, even if parallel isn't active today.

Document Version: 2.0 — Expert-Reviewed Created: 2026-04-03 Reviews: Workflow Systems Expert, Procurement Governance Expert Status: Ready for Customer Discussion